

MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE RUSSIAN
FEDERATION
FEDERAL STATE AUTONOMOUS EDUCATIONAL INSTITUTION OF HIGHER
EDUCATION
"NOVOSIBIRSK NATIONAL RESEARCH UNIVERSITY
STATE UNIVERSITY"
(NOVOSIBIRSK STATE UNIVERSITY, NSU)

15.03.06 - Mechatronics and Robotics

Focus (profile): Artificial Intelligence

EXPLANATORY NOTE

Job topic:

‘DINO’

Макогон Полина
Браун Анастасия
Брюханов Александр

Novosibirsk

2026

1. ТЕРМИНЫ И АББРЕВИАТУРЫ	3
2. ВВЕДЕНИЕ	4
3. НАЗНАЧЕНИЕ И ПРИМЕНЕНИЕ	4
4. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	4
4.2.1 Игровой дисплей	4
4.2.2 Контроллер состояния игры	5
4.2.3 Контроллер персонажа	6
4.2.4 Модуль подключения клавиатуры	7
4.2.5 Контроллер препятствий	8
4.2.6 Обработка коллизий	10
4.2.7 Счетчик очков	10
4.2.9 Модуль таблицы рекордов	11
4.2.10 Интеграция процессора и памяти	12
4.2.11 Организация памяти	13
5 ФУНКЦИОНАЛЬНЫЕ ХАРАКТЕРИСТИКИ	13
6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ	14
7 ЗАКЛЮЧЕНИЕ	14
8 ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ	15
9. ПРИЛОЖЕНИЕ	16

1. ТЕРМИНЫ И АББРЕВИАТУРЫ

CdM-8	Процессор Coco-de-Mer 8
I/O	Ввод / вывод
Harvard architecture	Архитектура с раздельной памятью данных и инструкций

2. ВВЕДЕНИЕ

Проект представляет собой аппаратно-программную игровую систему, реализованную в среде Logisim с использованием процессора CdM-8 Mark 5 Processor и программирования на языке ассемблера.

Работа выполнена в рамках учебной дисциплины, связанной с цифровыми платформами и архитектурой вычислительных систем.

3. НАЗНАЧЕНИЕ И ПРИМЕНЕНИЕ

«DINO» представляет собой аркадную игру, в которой игрок управляет персонажем, преодолевающим препятствия.

Основная цель игрока — как можно дольше избегать проигрыша.

Проект демонстрирует:

- работу процессора в составе вычислительной системы;
- организацию памяти;
- взаимодействие аппаратной и программной частей;
- реализацию игровой логики с использованием цифровых схем и ассемблерного кода.

Проект может использоваться в образовательных целях при изучении:

- цифровой схемотехники;
- низкоуровневого программирования;
- организации памяти.

4. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

Проект реализован в виде самостоятельной вычислительной системы, включающей: процессор, память, игровые контроллеры, дисплеи, модули ввода.

Архитектура системы объединяет аппаратную логику и программные инструкции, выполняемые процессором.

4.2.1 Игровой дисплей

Игровой дисплей представляет собой 6 матриц светодиодов (3 больших, 3 маленьких), отображающих:

- персонажа;
- препятствия;

- игровое пространство.

Игровые объекты отображаются посредством побитового управления строками.

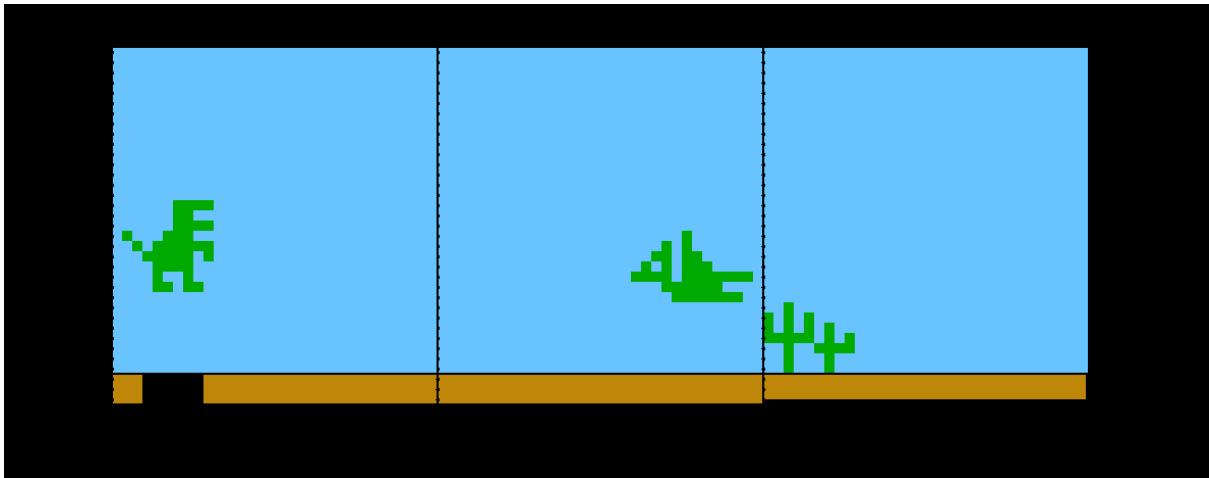


рис.1 “игровой дисплей”

4.2.2 Контроллер состояния игры

Контроллер состояния игры управляет тремя основными состояниями:

- restart;
- init - предзагрузка таблицы рекордов;
- game_over.

При запуске игры активируются процессор, дисплей - после инициализации таблицы. При game_over препятствия на дисплеи сменяются надписью “GAME_OVER”. При рестарте на дисплей возвращаются препятствия (карта начинается с начала), исчезает “GAME_OVER”, очищаются очки игрока, обновляются жизни, записываются новые очки.

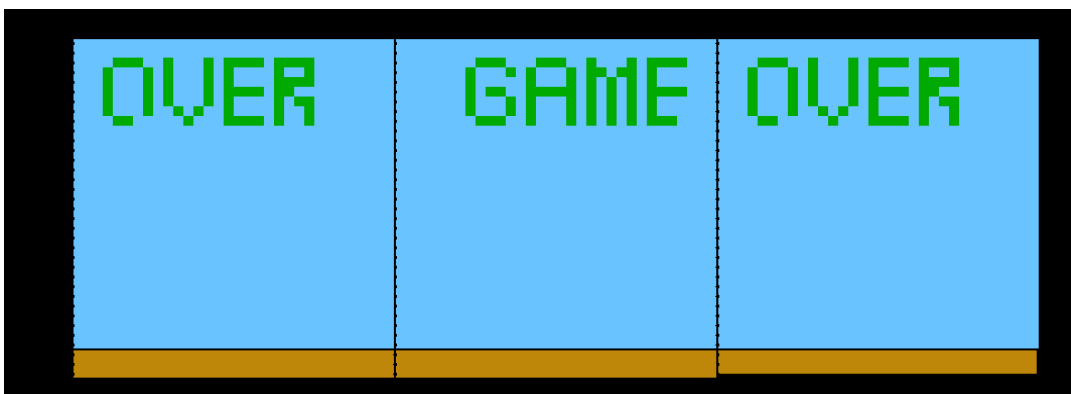


рис.2 “надпись game over”

При завершении игры происходит ожидание новой игры.

4.2.3 Контроллер персонажа

Контроллер персонажа отвечает за:

- прыжок/падение;

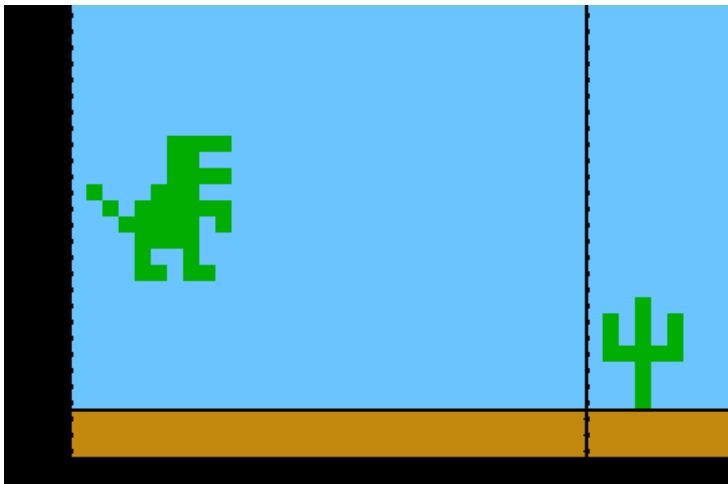


рис.3 “дино прыгает”

- смена спрайта;

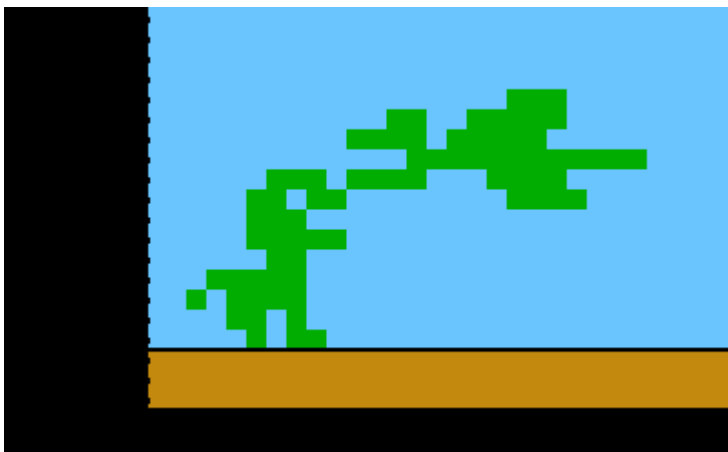


рис.4 “спрайт 2”

- движения ногами.
- приседание

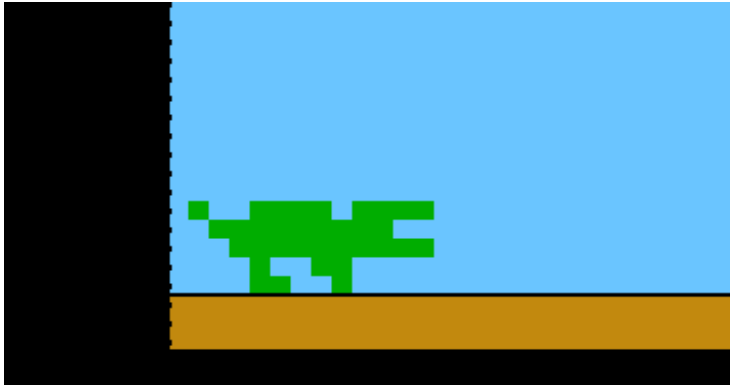


рис.5 “дино пригнулся”

Управление осуществляется через клавиатуру.

4.2.4 Модуль подключения клавиатуры

Модуль подключения клавиатуры обеспечивает взаимодействие пользователя с игрой.

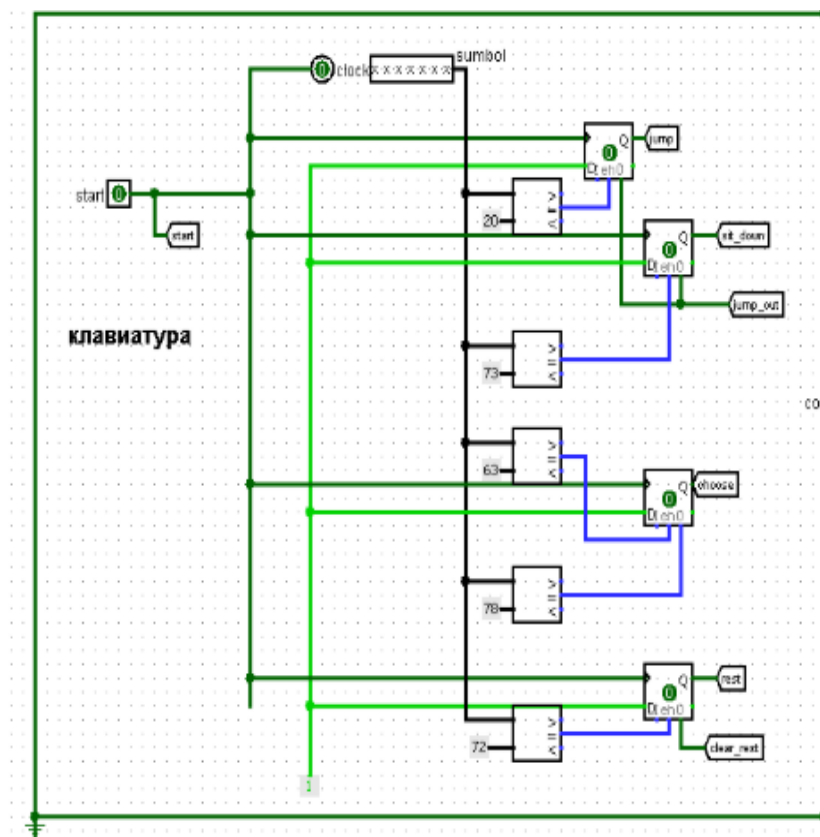


рис.6 “клавиатура”

Используемые клавиши:

Space	прыжок
s	пригиб
c	спрайт 2
x	спрайт 1
r	рестарт

4.2.5 Контроллер препятствий

1. чтение карты с ПЗУ

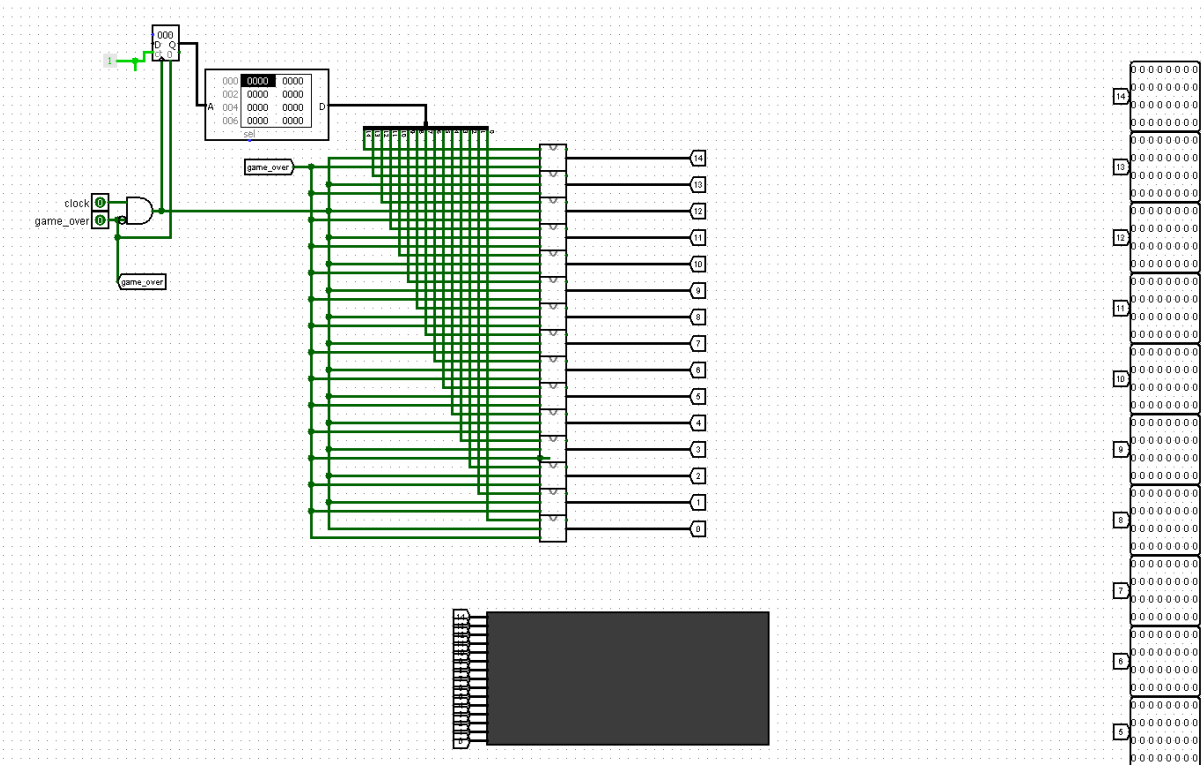


рис.7 “пример чтения карты с пзу”

2. запись на 1-ую матрицу (при game_over читается надпись “GAME_OVER”, иначе препятствия)

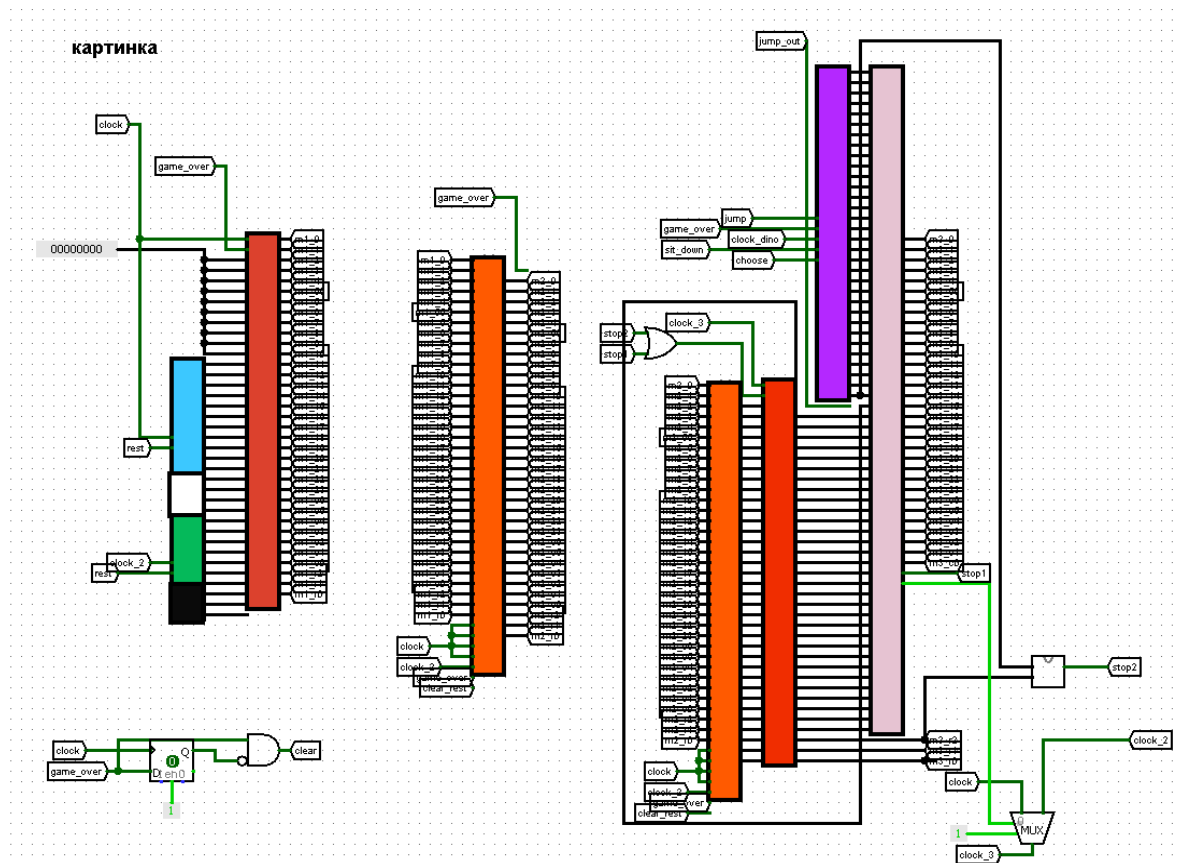


рис.8 “главная схема управления дисплеем”

3. сдвиг на следующие матрицы через сдвиговые регистры

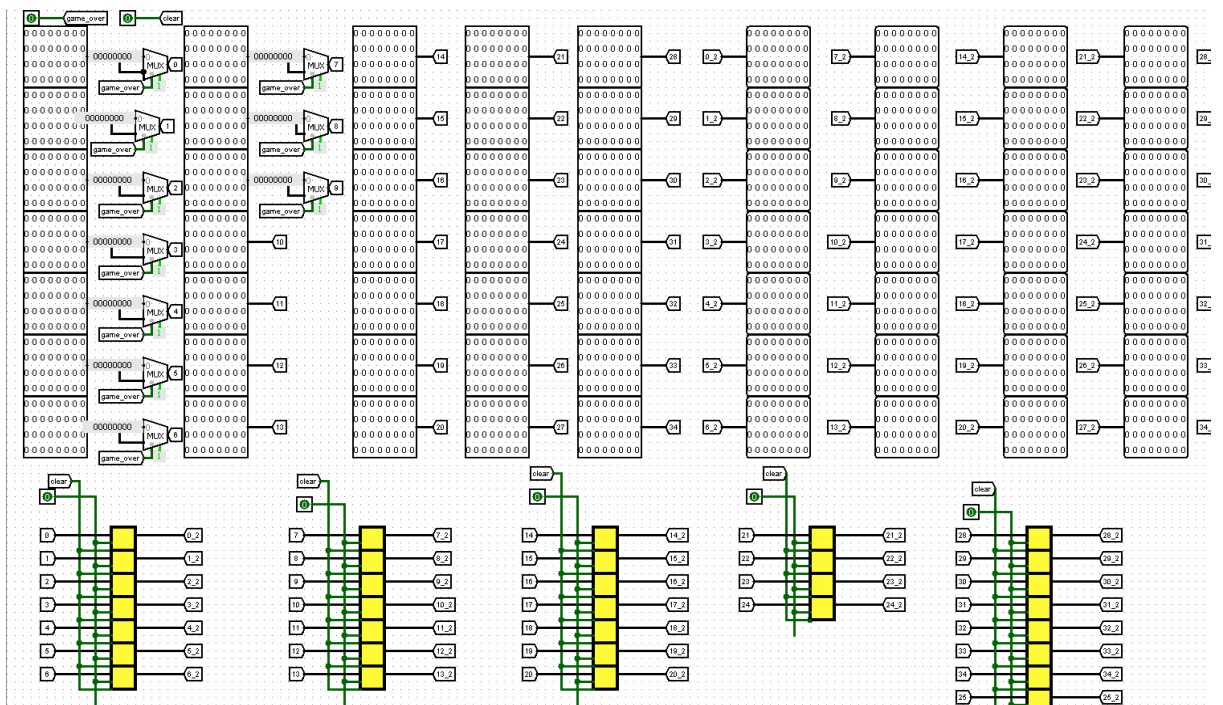


рис.9 “сдвиг карты на следующую матрицу”

Препятствия хранятся в отдельных ПЗУ (деление по 3 видам: верхние препятствия, нижние, надпись). Препятствия двигаются с разной скоростью (реализовано через делитель частоты).

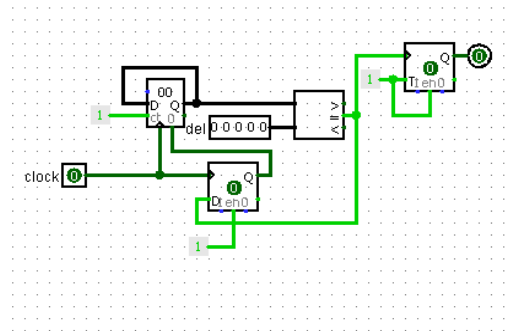


рис.10 “делитель частоты”

4.2.6 Обработка коллизий

Происходит проверка

- столкновение персонажа с верхним препятствием;
- столкновение с нижним препятствием

Для проверки столкновений схема коллизий принимает частоту, зависящую от типа препятствия

При столкновении с любым препятствием игра завершается.

4.2.7 Счетчик очков

Счётчик очков хранит текущее количество очков игрока. Очки начисляются за время игры. Максимальное значение счёта ограничено размером памяти и составляет 14 бит. После завершения игры счёт передаётся процессору для сравнения с рекордами.

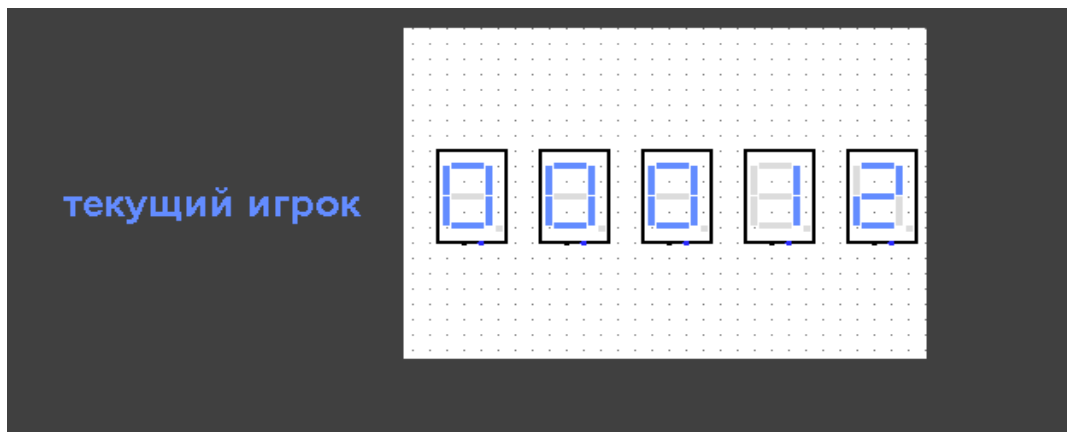


рис.11 “очки текущего игрока”

4.2.9 Модуль таблицы рекордов

Модуль таблицы рекордов хранит три лучших результата.



рис.12 “таблица рекордов”

При инициализации происходит предзагрузка таблицы лидеров

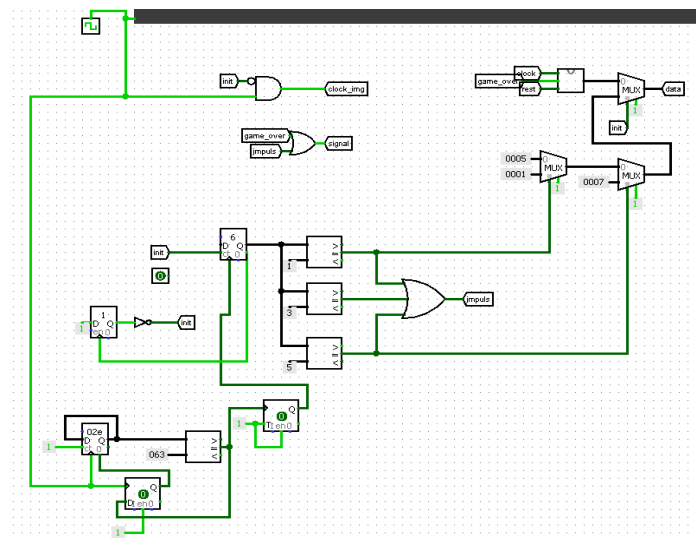


рис.13 “инициализация таблицы”

После завершения игры:

1. текущий результат записывается в память;
2. процессор сравнивает результат с существующими рекордами;
3. при необходимости происходит обновление таблицы.

Каждый результат хранится в двух байтах памяти:

- старший байт;
- младший байт.

4.2.10 Интеграция процессора и памяти

Процессор CdM-8 подключён к памяти через шину I/O, что позволяет передавать значения в обоих направлениях.

Используется Harvard architecture:

- память инструкций хранит код программы;
- память данных хранит результаты и игровые данные

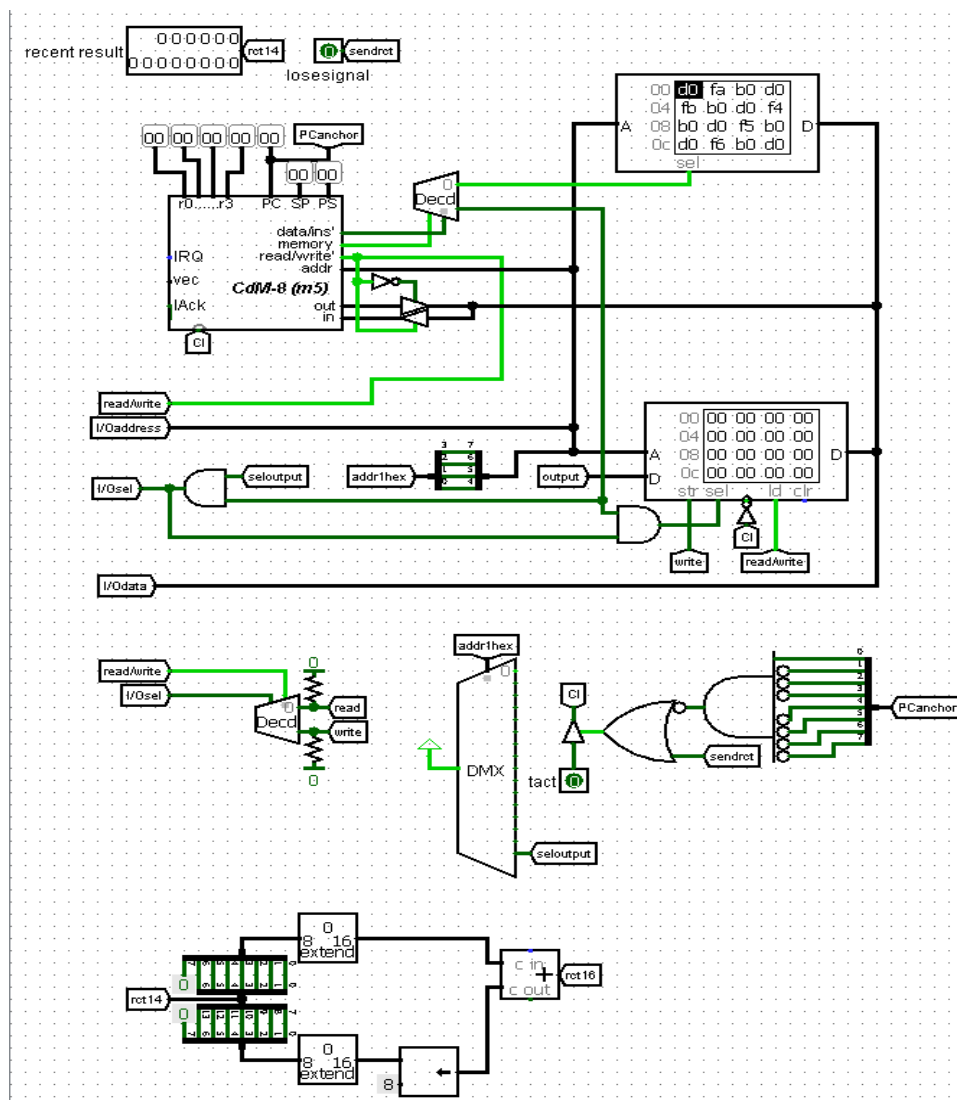


рис.14 “главная схема управления процессором”

Процессор циклически выполняет программу обновления рекордов. Чтение новых данных происходит при срабатывании сигнала “game_over” или “init”. Пока один из этих сигналов включен, цикл считывания будет работать.

Если новый балл достаточно велик, чтобы попасть в таблицу с лучшими тремя результатами, процессор начинает сортировку значений по возрастанию (одна итерация сортировки вставками).

4.2.11 Организация памяти

Для хранения рекордов используется область памяти начиная с адреса 0xF4. Текущий результат игрока также хранится в RAM.

Адрес	Назначение
0xF4	старший байт топ 1
0xF5	младший байт топ 1
0xF6	старший байт топ 2
0xF7	младший байт топ 2
0xF8	старший байт топ 3
0xF9	младший байт топ 3
0xFA	старший байт текущего игрока
0xFB	младший байт текущего игрока

К-во очков представляет собой 14-битное значение. Перед отправкой значение делится 2 байта памяти, которые отправляются независимо друг от друга с нулевым седьмым битом.

5 ФУНКЦИОНАЛЬНЫЕ ХАРАКТЕРИСТИКИ

Система поддерживает:

- движение персонажа;
- генерацию платформ;
- генерацию монет;
- подсчёт очков;
- столкновения;
- паузу;

- завершение игры;
- таблицу рекордов;
- сохранение результатов в памяти.

Программа процессора реализует:

- сравнение результатов;
- сортировку рекордов;
- обновление памяти.

Процессор полностью управляется программными инструкциями (см. приложение 1). Загруженный образ памяти в инструкционную память позволяет процессору пройти через процессы интеграции.

6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Игра начинается при подаче тактового сигнала. Управление осуществляется через клавиатуру (см. [4.2.4 Модуль подключения клавиатуры](#)). При проигрыше есть возможность начать новую игру через рестарт.

Главная схема - main. Над дисплеем (см. [4.2.1 Игровой дисплей](#)) находится блок отображения текущего к-ва жизней.



рис.15 “к-во жизней игрока”

Под дисплеем таблица рекордов (см. [4.2.9 Модуль таблицы рекордов](#))

7 ЗАКЛЮЧЕНИЕ

В ходе выполнения проекта была разработана полноценная аппаратно-программная игровая система.

Проект демонстрирует:

- интеграцию процессора в цифровую схему;
- работу с памятью;
- взаимодействие аппаратной и программной частей;
- использование ассемблерного кода для управления игровыми данными.

Система успешно выполняет:

- обработку игровой логики;
- обновление рекордов;
- управление отображением информации.

8 ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ

Бёрч, К. (2005). Документация. [онлайн] Logisim <https://cburch.com/logisim/ru/docs.html>

Приложение 1

исходный код программных инструкций

```
asect 0x00
```

```
# обновление рекордов
```

```
update:
```

```
    ldi r0, current_1
```

```
    ld r0, r0
```

```
    ldi r0, current_2
```

```
    ld r0, r0
```

```
    ldi r0, top1_1
```

```
    ld r0, r0
```

```
    ldi r0, top1_2
```

```
    ld r0, r0
```

```
    ldi r0, top2_1
```

```
    ld r0, r0
```

```
    ldi r0, top2_2
```

```
    ld r0, r0
```

```
    ldi r0, top3_1
```

```
    ld r0, r0
```

```
    ldi r0, top3_2
```

```
    ld r0, r0
```

```
br cmp_1
```

```
# сравнение с первым местом
```

```
cmp_1:
```

```
    ldi r0, top1_1
```

```
    ld r0, r0
```

```
    ldi r2, top1_2
```

```
    ld r2, r2
```

```
    ldi r1, current_1
```



```

ld r1, r1

if
    cmp r0, r1
is mi
    br replaceTop1 # текущий игрок стал первым
else
    if
        cmp r0, r1
    is z # сравниваем младший байт
        ldi r1, current_2
        ld r1, r1

        if
            cmp r2, r1
        is mi
            br replaceTop1 # текущий игрок стал первым
        else
            if
                cmp r2, r1
            is z
                br update # текущий игрок и так уже первый
            else
                br cmp_2 # сравнение со следующим местом
            fi
        fi
    else
        br cmp_2 # сравнение со следующим местом
    fi
fi

```

сравнение со вторым местом

cmp_2:

```

ldi r0, top2_1
ld r0, r0

ldi r2, top2_2
ld r2, r2

ldi r1, current_1
ld r1, r1

if
    cmp r0, r1
is mi
    br replaceTop2 # текущий игрок стал вторым
else
    if
        cmp r0, r1
    is z
        ldi r1, current_1
        ld r1, r1

```

```

        if
            cmp r2, r1
        is mi
            br replaceTop2 # текущий игрок стал вторым
        else
            if
                cmp r2, r1
            is z
                br update # текущий игрок и так уже второй
            else
                br cmp_3 # сравнение со следующим местом
            fi
        fi
    else
        br cmp_3 # сравнение со следующим местом
    fi
fi

# сравнение с третьим местом
cmp_3:
    ldi r0, top3_1
    ld r0, r0

    ldi r2, top3_2
    ld r2, r2

    ldi r1, current_1
    ld r1, r1

    if
        cmp r0, r1
    is mi
        br replaceTop3 # текущий игрок стал третьим
    else
        if
            cmp r0, r1
        is z
            ldi r1, current_2
            ld r1, r1

            if
                cmp r2, r1
            is mi
                br replaceTop3 # текущий игрок стал третьим
            else
                if
                    cmp r2, r1
                is z
                    br update # игрок уже третий либо не вошел в таблицу лидеров
                fi
            fi
        fi
    else
        br update # игрок не вошел в таблицу лидеров

```

fi

fi

вставка на первое место

replaceTop1:

```
ldi r0, top2_1
ld r0, r0
ldi r1, top3_1
st r1, r0
```

```
ldi r0, top2_2
ld r0, r0
ldi r1, top3_2
st r1, r0
```

```
ldi r0, top1_1
ld r0, r0
ldi r1, top2_1
st r1, r0
```

```
ldi r0, top1_2
ld r0, r0
ldi r1, top2_2
st r1, r0
```

```
ldi r0, current_1
ld r0, r0
ldi r1, top1_1
st r1, r0
```

```
ldi r0, current_2
ld r0, r0
ldi r1, top1_2
st r1, r0
```

br update

вставка на второе место

replaceTop2:

```
ldi r0, top2_1
ld r0, r0
ldi r1, top3_1
st r1, r0
```

```
ldi r0, top2_2
ld r0, r0
ldi r1, top3_2
st r1, r0
```

```
ldi r0, current_1
ld r0, r0
ldi r1, top2_1
```

```
st r1, r0
```

```
ldi r0, current_2
```

```
ld r0, r0
```

```
ldi r1, top2_2
```

```
st r1, r0
```

```
br update
```

```
# вставка на третье место
```

```
replaceTop3:
```

```
ldi r0, current_1
```

```
ld r0, r0
```

```
ldi r1, top3_1
```

```
st r1, r0
```

```
ldi r0, current_2
```

```
ld r0, r0
```

```
ldi r1, top3_2
```

```
st r1, r0
```

```
br update
```

```
br update
```

```
asect 0xF4
```

```
top1_1: ds 1 # старший байт 1-ого места
```

```
top1_2: ds 1 # младший байт 1-ого места
```

```
top2_1: ds 1 # старший байт 2-ого места
```

```
top2_2: ds 1 # младший байт 2-ого места
```

```
top3_1: ds 1 # старший байт 3-его места
```

```
top3_2: ds 1 # младший байт 3-его места
```

```
current_1: ds 1 # старший байт текущего игрока
```

```
current_2: ds 1 # младший байт текущего игрока
```

```
end
```